

Question 1. What is the difference between method overloading and method overriding? In which type of polymorphism does each belong?

Overloading: Methods share the same name but have different parameter lists. It is compile-time (static) polymorphism.

Overriding: A subclass reimplements an inherited method with the same signature. It is runtime (dynamic) polymorphism.

Question 2:

The output is "Bark!".

This is an example of method overriding and runtime (dynamic) polymorphism. Although variable `a` is declared as `Animal`, it refers to a `Dog` object. Therefore, Java calls the overridden `makeSound()` method in the `Dog` class at runtime.

Question 3. Why does Java NOT support multiple inheritance with classes? What is the "Diamond Problem"? How does Java solve this issue?

Java does not support multiple inheritance of classes to avoid ambiguity, such as the diamond problem, where a class inherits conflicting versions of a method from two parent classes. Java allows only one superclass but multiple interfaces, requiring conflicting default methods to be overridden.

Question 4:

The code fails to compile because `Shape` is an abstract class and cannot be instantiated directly. The statement `Shape s = new Shape();` causes a compilation error, so the program does not run or produce any output.

Question 5. What is the difference between an abstract class and an interface? Give one scenario where you would prefer using an interface over an abstract class.

An abstract class lets related classes share state and code, while an interface defines a contract that different classes can implement. A class can extend only one class but implement multiple interfaces. For example, `Printer` and `Camera` can both implement a `Connectable` interface to provide a `connect()` method. An interface is suitable because these devices share a capability but are different types of objects.

Question 6:

The output is "Animal eating" followed by "Meow!" on the next line.

Upcasting means treating a subclass object as a superclass type, as in `Animal a = new Cat();`. Downcasting explicitly casts a superclass reference to a subclass type, as in `Cat c = (Cat) a;`. The `instanceof` check ensures the cast is valid. Uncommenting `a.meow()` would cause a compilation error because `Animal` does not define `meow()`.

Question 7. What are the rules for overriding the equals() method? Why must we also override hashCode() when we override equals()?

The `equals()` method must be reflexive, symmetric, transitive, and consistent, and return false when comparing a non-null object to null. If we override `equals()` without overriding `hashCode()`, equal objects may have different hash codes, causing incorrect behavior in `HashSet` and `HashMap`. Therefore, we must override `hashCode()` to ensure equal objects have the same hash code.

Question 8. What is the difference between shallow copy and deep copy? Given an object Person with a field Address address, explain what happens to the address field in each type of copy.

A shallow copy creates a new Person object but copies the address reference, so both Person objects share the same Address object. Changes to that Address object affect both. A deep copy creates a new Person object and a separate copy of its Address object, so changes to one address do not affect the other.

Question 9:

The output is "Taking off from Flyable". Both interfaces define a default takeOff() method, so Duck overrides it to resolve the conflict. Flyable.super.takeOff() explicitly calls the default method from Flyable.

Question 10:

Methods A and B are valid overloads because A changes the number of parameters and B changes the parameter types. Methods C and D are invalid because they have the same parameter types as the original method. Changing only the return type, access modifier, parameter names, or method body does not create a valid overload. Therefore, the class as written fails to compile.